

Design Requirements

Project Name: Navigators

SIH Problem Statement ID: SIH26168

1. System Architecture

The system employs a 100% offline edge architecture focused entirely on edge inference to ensure real-time responsiveness and security when GNSS is denied. No backend or cloud dependency is allowed in the runtime.

1.1 Edge Engine (Smartphone / External IMU)

- **Sensor Interface:** Access raw data from smartphone sensors (Accelerometer, Gyroscope) at 10 Hz. Supports up to 200 Hz for external FOG-IMU.
- **Preprocessing Module:** Employs non-linear median filtering and Butterworth low-pass filtering to isolate and remove physical shocks (e.g., potholes) without distorting kinematic data. Calculates phone-to-vehicle alignment.
- **AI Inference Engine:** A lightweight neural network (Temporal Convolutional Network or LSTM) trained with rigorous data augmentations (noise injection, dynamic scaling). Designed to predict 2D velocity from windowed IMU data.
- **Sensor Fusion (EKF):** A 15-state Extended Kalman Filter implemented entirely in JavaScript to optimally fuse AI-predicted velocities with GNSS readings (when available) directly on the edge.
- **Kinematic Constraints Module:** Enforces Non-Holonomic Constraints (NHC) and Zero Velocity Updates (ZUPT) dynamically.
- **Map Matching Module:** Snaps estimated diverging trajectories back to known road centerlines using Geometric and Probabilistic (HMM) offline mapping.
- **UI/Visualizer:** Renders the estimated trajectory continuously without jumping or freezing inside a Progressive Web App (PWA).

1.2 Processing Flow

1. **GNSS Available:** GNSS + IMU → AI/ML filtering → Fusion (EKF) → Map matching → Position
 2. **GNSS Lost:** IMU → AI speed/motion estimation → Dead reckoning → NHC & ZUPT → Map matching → Position
 3. **GNSS Returns:** Dead reckoning → Re-acquisition → GNSS + INS fusion → Corrected continuous position
-

2. Software Requirements

- **AI/ML Framework:** PyTorch for training; ONNX Runtime Web for 100% offline edge deployment inside the browser.
 - **Data Processing:** Pandas, NumPy, SciPy for preprocessing and filtering.
 - **Navigation Engine:** Pure JavaScript implementation for EKF, NHC, ZUPT, and Map Matching to eliminate Python backend dependencies during deployment.
-

3. Hardware Requirements

- **Development:** Workstation or Cloud instance with GPU (NVIDIA) for training on the large IO-VNBD dataset.
- **Deployment:** Modern smartphone (Android/iOS) with capable IMU, utilizing the CPU/NPU for WebAssembly inference, or dedicated edge hardware for external IMU integration.